

ASSESSMENT DECLARATION COVER SHEET

Please complete this cover sheet for each assessment submission. For group assessments, each group member must individually complete and submit a cover sheet.

STUDENT (REGISTRATION) NUMBER: 10009023

MODULE TITLE: Honours Project (GA)

DATE OF SUBMISSION: 08/05/2026

I declare that, except where explicitly acknowledged, this assessment is my own work and has not been submitted for any other module or degree programme at Edinburgh Napier University or any other institution.

This declaration is made in compliance with Edinburgh Napier University's Academic Integrity Regulations.

Use of Generative AI Tools

☒ **Yes**, I have used Gen AI tools for this submission and I have described how below.

☐ No, I have not used Gen AI tools for this submission.

Description of use:

I leveraged AI during the preliminary research phase to identify relevant academic papers and conferences. Given the niche nature of the subject matter, this proved to be an effective starting point for sourcing materials. Additionally, I sought assistance with LaTeX formatting, particularly in resolving challenges related to figure presentation.

Edinburgh Napier University

School of Computing, Engineering and the Built Environment

SOC10405 – Honours Project (GA)

An Investigation and Implementation of Rust as a Cross-Platform GUI Framework

Interim Report

Timothy Graham

Student Number: 10009023

10009023@napier.ac.uk

Supervisor: Sally Smith

8th May 2026

Abstract

Cross-platform graphical user interface (GUI) development is currently dominated by frameworks built on web technologies, most notably React Native, Flutter, and Electron. Whilst these solutions provide broad platform reach, they do so at the cost of native performance, deterministic memory behaviour, and compile-time correctness guarantees. This project investigates whether Rust, a systems programming language with a formally verified ownership-and-borrowing type system and no garbage collector, can serve as the foundation for a viable cross-platform native GUI framework. The work takes a development-led approach: a lightweight GUI framework will be designed and implemented in Rust, building upon the GPU-accelerated GPUI library released by Zed Industries, and validated through the construction of two concrete applications, namely a calculator and a memo application, targeting Linux, macOS, and Windows. AI-assisted code generation will be used to produce equivalent implementations in React Native, Flutter, and Electron, providing a consistent and reproducible baseline for comparison across rendering performance, binary size, memory consumption, and developer experience.

Contents

1	Introduction and Problem Statement	2
1.1	Project Context	3
1.2	Aims and Objectives	3
2	Literature Review	4
2.1	The Case for Rust in Systems-Level UI Development	4
2.2	Formal Foundations of Rust’s Safety Guarantees	5
2.3	Rust’s Performance Characteristics	6
2.4	Developer Experience and the Learning Curve	6
2.5	The Existing Rust GUI Ecosystem	7
2.6	Cross-Platform GUI Frameworks: The Incumbent Landscape	8
2.7	The Role of AI-Assisted Code Generation	8
2.8	WebAssembly and the Broader Rust Frontend Landscape	9
2.9	Summary of Literature Review	10
3	Methodology	10
3.1	Research Approach	10
3.2	Framework Design	11
3.3	Application Demonstrations	12
3.4	Use of AI-Assisted Code Generation	14
3.5	Evaluation Plan	14
4	Plan of Work	16
5	Risk Assessment	18
6	Supervision Meetings	19
7	Conclusion	19

1 Introduction and Problem Statement

Cross-platform GUI development is a problem that has existed as long as there have been multiple desktop operating systems. Applications must behave consistently and perform well across Linux, macOS, and Windows, ideally from a single codebase. The dominant industry responses to this challenge each make significant trade-offs.

Electron, which underpins applications such as Visual Studio Code and Slack, achieves cross-platform compatibility by bundling a full Chromium rendering engine alongside a Node.js runtime. This results in large binary sizes, elevated memory consumption, and UI responsiveness that is constrained by JavaScript execution and garbage-collection pauses (Lissens, 2023). React Native and Flutter adopt more performant rendering models, but remain rooted in managed-runtime languages, JavaScript and Dart, respectively, which impose garbage-collection overhead and provide weaker static correctness guarantees than systems languages (Haas et al., 2017).

Rust, stabilised in 2015 and now governed by the independent Rust Foundation, was designed to address precisely this gap. It provides C-level performance and direct memory control without requiring manual memory management, enforcing correctness through a compile-time ownership and borrowing type system that eliminates use-after-free errors, data races, and null-dereference bugs by construction (Matsakis and Klock, 2014; Jung et al., 2020). These properties have driven rapid adoption in operating systems, embedded firmware, and network infrastructure. A significant signal of the language's maturity came in December 2022 when Rust became the second programming language, after C, to be officially accepted into the Linux kernel (The Linux Kernel Organization, 2022). This was not a minor addition: it followed years of debate among kernel maintainers and represented a formal acknowledgement that Rust's safety guarantees are strong enough for use in the most widely deployed operating system kernel in the world. This decision has since accelerated Rust's adoption across the wider systems programming community.

In user-interface development, the most visible example of Rust's potential is Zed, a code editor written entirely in Rust by the creators of Atom and Tree-sitter. Zed ships with GPUI, an open-source GPU-accelerated User Interface framework licenced under Apache 2.0 (Zed Industries, 2024). GPUI renders all interface elements directly to the GPU using Metal on

macOS and Vulkan or DirectX 12 on other platforms, targeting a consistent 120 frames per second (Zed Industries, 2025c). This represents a fundamentally different architecture to Electron, and the performance difference is perceptible in everyday use.

Despite this industrial momentum, there is no published academic study that empirically investigates the feasibility and performance characteristics of using Rust as the foundation for a cross-platform GUI framework in the tradition of React Native or Flutter. This project addresses this gap.

1.1 Project Context

This is a development-led honours project. Its primary output is a software deliverable: a cross-platform GUI framework built in Rust on top of GPUI, together with two demonstration applications that validate the framework's design. The project also produces an empirical comparison of the Rust framework against React Native, Flutter, and Electron across a defined set of performance and developer-experience metrics.

The literature review that follows establishes the academic context for both the language choice and the framework design decisions.

1.2 Aims and Objectives

The primary aim of this project is to investigate and implement Rust as the basis for a cross-platform GUI framework, and to evaluate its practical viability against established alternatives.

The specific objectives are:

1. To conduct a critical review of the literature on Rust's memory safety, performance, and concurrency properties, and of existing cross-platform GUI frameworks.
2. To design a lightweight cross-platform GUI framework in Rust, using GPUI as the underlying rendering and windowing layer, targeting Linux, macOS, and Windows from a single codebase.
3. To implement two demonstration applications using the framework: a calculator application and a memo application, chosen to exercise a range of common UI primitives including input handling, state management, layout, and text rendering.

4. To produce equivalent implementations of both applications in React Native, Flutter, and Electron, using AI-assisted code generation as a consistent and reproducible methodology.
5. To evaluate all implementations across rendering frame rate, startup latency, binary size, resident memory consumption, and developer effort.
6. To identify the limitations, practical barriers, and future development directions for Rust-based cross-platform GUI development.

2 Literature Review

This literature review is structured thematically, moving from the broad properties of Rust that motivate its use in GUI development, through the formal foundations of those properties, to the specific landscape of cross-platform GUI frameworks against which this project's output will be evaluated.

2.1 The Case for Rust in Systems-Level UI Development

The motivation for investigating Rust as a GUI language begins with the documented limitations of the incumbent approaches. Szekeres et al. (2013) provide a systematic analysis of memory safety vulnerabilities in C and C++ codebases, identifying use-after-free errors, buffer overflows, and data races as the dominant root causes of exploitable defects. This finding is corroborated at scale by the Microsoft Security Response Centre, which reports that approximately 70% of all security vulnerabilities it addresses across its products are attributable to memory safety failures (Microsoft Security Response Centre, 2019). These observations establish that C and C++, whilst performant, carry a safety cost that is practically difficult to eliminate through programmer discipline alone.

Rust's response to this problem is architectural. Matsakis and Klock (2014) describe the language's ownership model: every value has a single owner, ownership can be moved but not copied by default, and mutable aliasing is statically prohibited by the borrow checker. This eliminates the root causes identified by Szekeres et al. (2013) at compile time, without any runtime overhead. Bugden and Alahmar (2022) benchmark Rust against C, C++, Go, Java, and Python across both safety and performance dimensions. They find that Rust achieves performance comparable to C and C++ whilst being the most safe language tested,

particularly in concurrent scenarios where its data-race prevention is most consequential.

The acceptance of Rust into the Linux kernel in December 2022 is a particularly significant marker of the language’s maturity (The Linux Kernel Organization, 2022). The Linux kernel is one of the most scrutinised codebases in existence, and the decision to permit Rust as a second implementation language followed an extensive review by the kernel maintainers. Linus Torvalds himself approved the initial Rust infrastructure, noting that the safety properties Rust provides are relevant to a codebase where memory errors have historically caused security vulnerabilities (Torvalds, 2022). This acceptance demonstrates that Rust’s safety model is considered sound enough for production use in the most demanding systems environment, which provides strong justification for its use as a GUI framework foundation.

For GUI development specifically, the absence of a garbage collector is important. McCarthy et al. (2022) demonstrate empirically that garbage-collection pauses cause measurable latency degradation in GUI applications. In interactive applications, these pauses manifest as visible frame drops. Rust’s deterministic memory reclamation, governed entirely by scope and ownership rules rather than a runtime collector, eliminates this source of inconsistency and enables the kind of consistent, high-frequency rendering that GPUI targets.

2.2 Formal Foundations of Rust’s Safety Guarantees

The safety properties described above have received rigorous formal treatment. Jung et al. (2020) present RustBelt, a machine-checked proof in the Coq proof assistant that well-typed safe Rust programmes cannot exhibit undefined behaviour arising from memory unsafety. The proof is constructed using the Iris separation logic framework and covers a substantial fragment of the Rust type system, including the borrow checker’s aliasing rules. There is no equivalent formal result for C++.

Kan et al. (2018) develop K-Rust, an executable formal semantics for Rust’s ownership system using the K-Framework, with machine-checked proofs of the memory safety properties it enforces. Lattuada et al. (2023) extend these foundations further with Verus, an SMT-based tool for verifying functional correctness properties of Rust programmes beyond what the type system alone can assert. Taken together, this body of work establishes that Rust’s safety guarantees are formally verified rather than aspirational, which is directly relevant to the reliability of a GUI framework intended for deployment across multiple operating systems.

The ownership model also has implications for concurrent GUI programming. Rust's Send and Sync marker traits extend ownership semantics to threads: the compiler statically verifies that values crossing thread boundaries are safe to do so, and refuses to compile code that would produce data races (Klabnik and Nichols, 2019). As Bugden and Alahmar (2022) note, this makes Rust the most safe language in concurrent environments, which is directly relevant to GUI frameworks that must manage input events, network requests, and rendering operations simultaneously.

2.3 Rust's Performance Characteristics

Ivanov (2022) benchmark Rust against C++ using commonly-used algorithms and data structures, finding overall performance to be comparable with only minor disadvantage in certain operations. This parity is achieved without manual memory management, through what Klabnik and Nichols (2019) describe as zero-cost abstractions: high-level constructs such as iterators, closures, and trait objects compile to machine code that is equivalent in quality to careful hand-written C.

For rendering workloads, the GPU pipeline approach used by GPUI is relevant. Using low-level graphics APIs such as Vulkan and Metal directly, rather than routing through a CPU-side widget toolkit, allows the GPU to operate on interface geometry without the bottlenecks introduced by CPU-to-GPU layer compositing (Zed Industries, 2025c). LogRocket Blog (2024) document that this architecture allows Zed to achieve a startup time and frame rate that are perceptibly faster than Electron-based editors such as Visual Studio Code, which is consistent with the theoretical expectation from eliminating a full Chromium embedding.

2.4 Developer Experience and the Learning Curve

The theoretical advantages of Rust must be weighed against the documented challenges of adopting it. Crichton et al. (2022) conduct a mixed-methods study examining 100 Stack Overflow questions and surveying 101 Rust programmers, finding that ownership and lifetime rules are the primary barriers to learning and productive use. The borrow checker, whilst formally sound, produces compiler errors that can be difficult to interpret, particularly for developers accustomed to garbage-collected languages. Ferdowsi (2023) survey this empirical literature and conclude that Rust's type system comes with a steep learning curve that has been repeatedly cited as a barrier to adoption.

Fulton et al. (2021) offer a more nuanced view through a qualitative study with 16 professional Rust developers. Whilst the borrow checker is identified as the primary source of friction, it is also valued for catching bugs that would otherwise reach production. Developers report that after an initial adjustment period productivity recovers and the language’s guarantees provide confidence during refactoring. In the context of this project, developer experience is not merely a practical concern but an explicit evaluation dimension. One objective is to document whether building a GUI framework in Rust involves qualitatively different and measurably more difficult patterns than equivalent work in JavaScript or Dart.

2.5 The Existing Rust GUI Ecosystem

Before justifying the framework design decisions, it is necessary to survey what already exists in the Rust GUI landscape. Horn (2025) conducts the most comprehensive recent audit, evaluating 43 Rust GUI crates against three criteria: basic functionality, screen-reader accessibility, and IME input handling. The assessment is sobering: approximately 94% of surveyed libraries were found not to be production-ready, with build failures, missing features, or active abandonment being common. This finding directly motivates the decision to build upon GPUI rather than starting from scratch.

GPUI is described by its authors as a hybrid immediate and retained mode, GPU-accelerated UI framework for Rust designed to support a wide variety of applications (Zed Industries, 2025d). It is the rendering engine that powers the Zed editor, which Heise Online (2026) describe as being built like a video game, rendering all interface elements directly to the GPU on every frame. GPUI is licensed under Apache 2.0, making it suitable for academic and commercial derivative work, and it has attracted a growing ecosystem of third-party applications including database clients, terminal emulators, and productivity tools (Zed Industries, 2025a). It correctly handles IME input and has been identified as one of the few Rust GUI libraries that meets basic internationalisation requirements (Horn, 2025).

Beyond GPUI, the ecosystem includes egui (Ernerfeldt, 2024), an immediate-mode library requiring no external dependencies and supporting WebAssembly compilation; Iced, an Elm-inspired retained-mode framework; Slint, which provides a domain-specific language for UI declarations; and Dioxus, a React-inspired framework targeting web, desktop, and mobile. Each represents a distinct design philosophy, and their comparative strengths are discussed in the context of the framework design decisions in Section 3.

2.6 Cross-Platform GUI Frameworks: The Incumbent Landscape

Three frameworks have been selected for comparison on the basis of their industry prevalence and their representativeness of different architectural approaches.

React Native is a cross-platform framework developed by Meta that renders native platform UI components using a JavaScript bridge (Meta Platforms, Inc., 2025). Its core strength is code sharing with web-based React applications and familiarity with the JavaScript and TypeScript ecosystem. Its primary documented limitation is the overhead of the bridge between the React component tree and the native rendering layer, which introduces latency in high-frequency UI updates.

Flutter, developed by Google, takes a different approach. Rather than mapping to native UI components, it provides its own rendering engine and draws all interface elements itself (Google LLC, 2025). Flutter applications are compiled to native code via the Dart AOT compiler, though the Dart runtime uses a generational garbage collector. As McCarthy et al. (2022) show, GC pauses remain a source of latency variance in managed-runtime frameworks including Flutter, making it a useful point of comparison for Rust's deterministic memory model.

Electron combines a Chromium rendering engine with a Node.js backend, enabling web technologies to be used for desktop application development (OpenJS Foundation, 2025). Its principal advantage is developer familiarity and access to the npm ecosystem. Its principal disadvantage is resource consumption: binary sizes typically exceed 100 MB and baseline memory usage is substantially higher than native applications due to the bundled browser engine (Lissens, 2023). The Zed team cite these constraints as the explicit motivation for moving away from an Electron-based architecture (LogRocket Blog, 2024).

These three frameworks cover the dominant architectural patterns in cross-platform GUI development: native-component bridging (React Native), own-renderer with managed runtime (Flutter), and embedded browser engine (Electron).

2.7 The Role of AI-Assisted Code Generation

A methodological consideration for this project is the use of AI-assisted code generation to produce the comparative framework implementations. The use of AI tools for code generation

was discussed and agreed with the project supervisor as an appropriate methodology for producing consistent, reproducible baseline implementations in React Native, Flutter, and Electron. The agreed approach is that the Rust framework and demonstration applications will be implemented by hand, reflecting the primary research contribution of the project. The equivalent implementations in the comparison frameworks will be generated using a state-of-the-art AI coding tool, with the prompts, outputs, and any manual corrections fully documented.

This approach has two benefits beyond reducing implementation effort. First, it provides a consistent methodology for producing the comparison code: the same specification will be used as the basis for each generated implementation, reducing the risk that differences in developer familiarity with each framework introduce bias into the comparison. Second, the quality and completeness of the AI-generated implementations across frameworks is itself an informative data point, as it reflects how well each framework’s idioms and conventions are represented in current AI training data.

The use of AI tools in this project is disclosed in accordance with Edinburgh Napier University’s academic integrity guidelines.

2.8 WebAssembly and the Broader Rust Frontend Landscape

Whilst this project focuses on native desktop GUI development, the parallel trajectory of Rust in browser-based interfaces via WebAssembly provides useful context. Haas et al. (2017) present the WebAssembly standard as a binary instruction format enabling near-native execution in browsers, and Watt (2018) provide a machine-checked proof of WebAssembly’s type soundness. Empirical comparisons confirm performance advantages for WebAssembly in compute-intensive workloads: De Macedo et al. (2022) find WebAssembly to be up to 20.88% more energy-efficient and 5.15% faster than equivalent JavaScript in Google Chrome. These findings are relevant context for understanding the breadth of interest in Rust as a UI language, even though the primary focus of this project is the native desktop case.

2.9 Summary of Literature Review

The literature establishes three core justifications for this project. First, Rust’s ownership model provides formally verified memory safety and data-race freedom at compile time (Jung et al., 2020; Kan et al., 2018), addressing the primary weakness of C and C++ toolkits and eliminating the GC-induced latency that affects Flutter and JavaScript-based frameworks (McCarthy et al., 2022). Second, Rust’s performance is empirically comparable to C++ (Ivanov, 2022) and substantially exceeds managed-runtime alternatives in latency-sensitive workloads, with GPUI demonstrating GPU-direct rendering that is unachievable by Electron (Zed Industries, 2025c; LogRocket Blog, 2024). Third, the acceptance of Rust into the Linux kernel confirms the language’s maturity at the systems level (The Linux Kernel Organization, 2022), and GPUI provides a technically sound foundation for the GUI work (Horn, 2025; Zed Industries, 2025d). The gap this project addresses, namely a focused and academically evaluated cross-platform framework built on these foundations, is not covered by existing literature.

3 Methodology

3.1 Research Approach

This project follows a design-science research methodology (Hevner et al., 2004), in which the primary research contribution is a software deliverable: the cross-platform GUI framework and its demonstration applications. The deliverable is evaluated against functional requirements derived from the literature and comparative benchmarks provided by the incumbent frameworks. This approach aligns directly with the development-led honours project model, in which the literature review identifies the problem and the existing solutions, and the implementation and evaluation provide the contribution.

Figure 1 shows the overall structure of the project, from the initial framework implementation through to the final comparative evaluation.

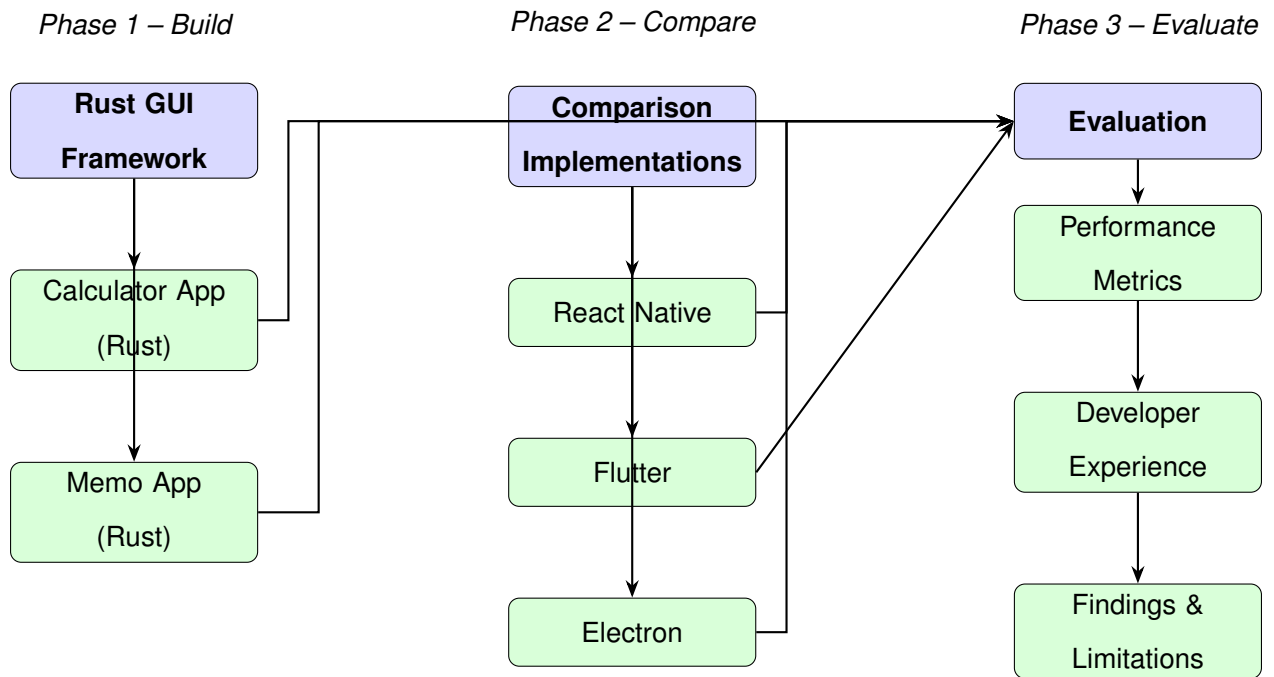


Figure 1: Overview of the project structure across the three main phases

3.2 Framework Design

The framework will be implemented in Rust, using GPUI as the underlying rendering and windowing layer. GPUI was selected over the alternatives identified in Section 2.5 for the following reasons. Unlike egui's immediate-mode model, GPUI's hybrid immediate/retained architecture allows efficient partial updates without rebuilding the entire scene graph on every frame. Unlike Slint's domain-specific language approach, GPUI exposes a Rust-native API, ensuring that the framework's type safety guarantees extend throughout application code. Unlike Dioxus's WebView-backed desktop target, GPUI renders via native GPU APIs, specifically Metal, Vulkan, and DirectX 12, delivering the performance profile this project aims to investigate.

Figure 2 illustrates the layered architecture of the framework.

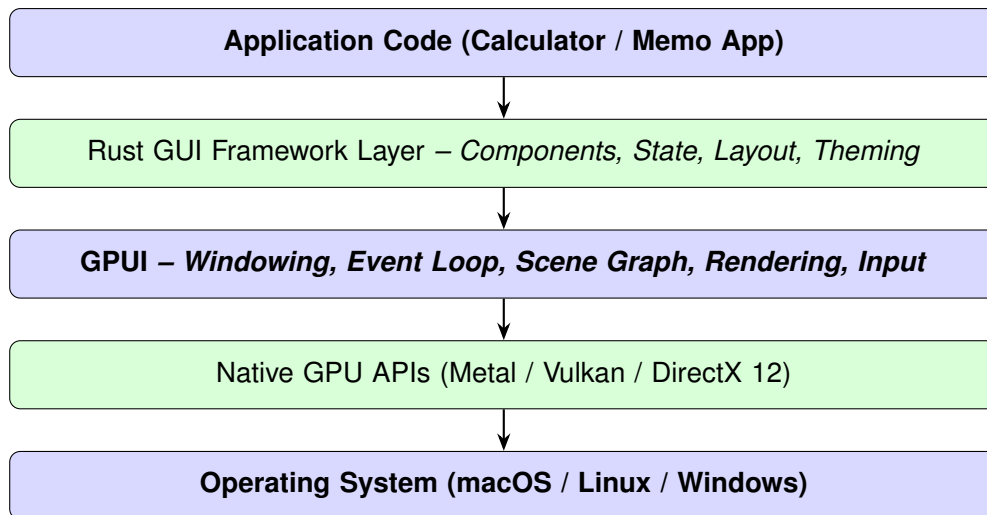


Figure 2: Layered architecture of the Rust GUI framework

The framework layer built on top of GPUI will provide the following components:

- A **component abstraction** for composing UI elements in a reusable, declarative style.
- A **reactive state management model** using signals, in which state changes propagate only to the components that depend on them.
- A **layout engine** supporting common box-model primitives, allowing components to position and size themselves relative to their containers.
- A **theming system** that allows visual properties such as colour, typography, and spacing to be defined centrally and applied consistently.
- **Platform-specific window management** to handle the differences in window creation and event handling between Linux, macOS, and Windows.

The scope is deliberately constrained. This is a research vehicle designed to validate the architectural approach and produce meaningful evaluation data, not an attempt to produce a production-grade framework comparable to Flutter or Qt.

3.3 Application Demonstrations

Two applications will be implemented using the framework. Figure 3 shows the intended user flow for each.

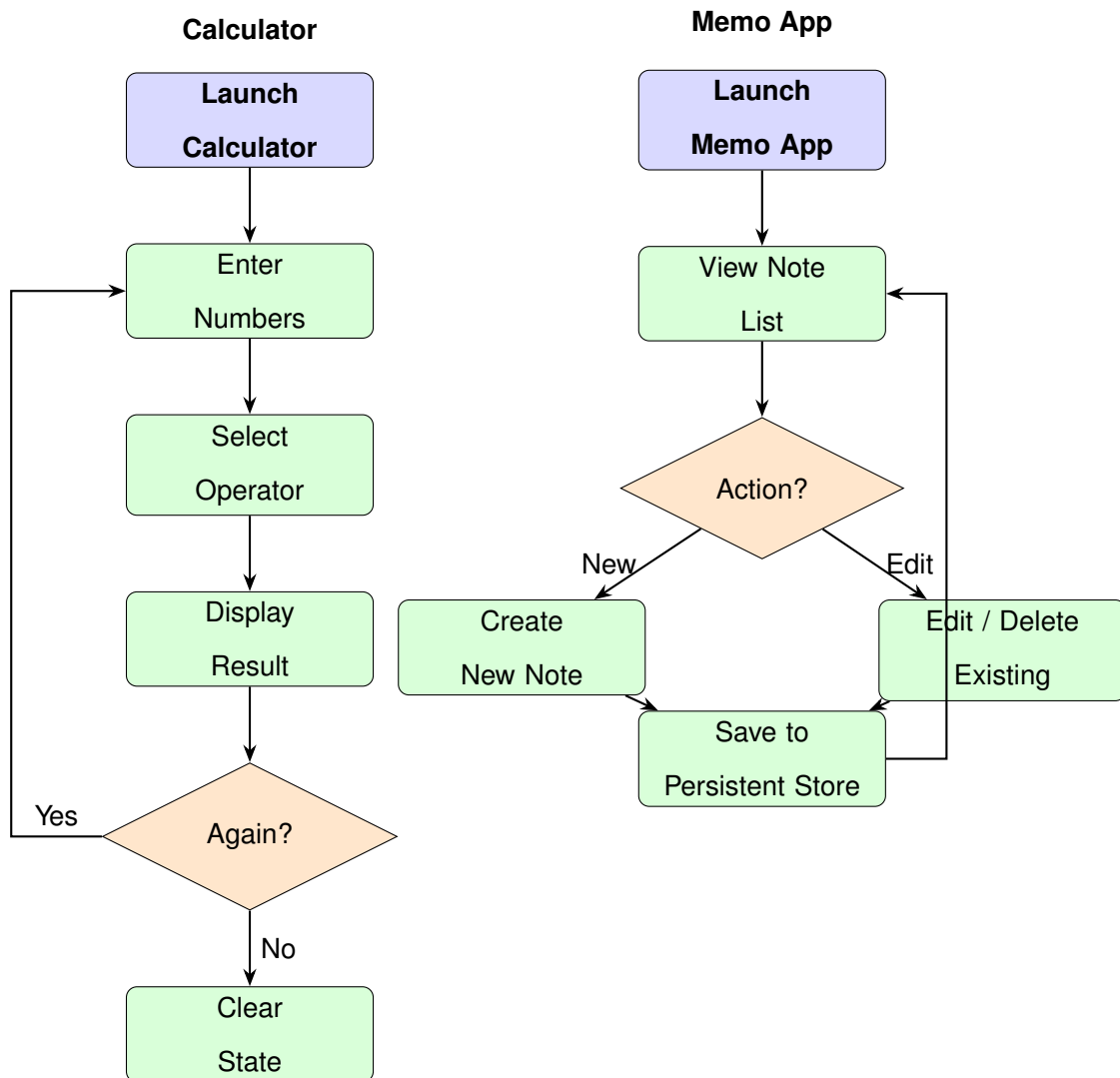


Figure 3: User flow diagrams for the calculator and memo applications

Calculator application: A standard four-function calculator with a numeric keypad, display panel, and operator buttons. This application is deliberately simple in scope so that the evaluation can focus on the framework’s rendering pipeline, input handling, and state management rather than application logic. The same specification will be used to produce equivalent implementations in React Native, Flutter, and Electron.

Memo application: A basic note-taking application supporting the creation, editing, and deletion of text notes with persistent storage. This exercises a broader range of UI primitives than the calculator, including multi-line text input, a scrollable list, and state that persists between application sessions. The increased complexity allows the framework to be tested more thoroughly against realistic use cases.

Both applications will target Linux, macOS, and Windows, and the framework will be designed from the outset to compile and run correctly on each platform.

3.4 Use of AI-Assisted Code Generation

As agreed with the project supervisor, AI-assisted code generation will be used to produce the comparison implementations in React Native, Flutter, and Electron. The rationale and disclosure for this are described in Section 2.7. The practical workflow is as follows.

A formal specification will be written for each application, describing the required UI elements, interactions, and data model in plain language. This specification will be submitted to the AI coding tool without modification for each target framework. The generated code will be tested, and any corrections required to make it compile and run correctly will be documented. The final comparison implementation will consist of the generated code plus any documented corrections.

This approach means that the comparison implementations reflect what an AI tool can produce from a clear specification, which is itself a useful data point. If the AI-generated implementations contain errors or require substantial correction for a particular framework, this is recorded as part of the developer-experience evaluation. The entire generation process, including prompts, outputs, and corrections, will be included as an appendix to the final report.

The Rust framework and demonstration applications are implemented by hand and represent the primary technical contribution of the project.

3.5 Evaluation Plan

The framework and applications will be evaluated across five dimensions. Figure 4 summarises the evaluation structure.

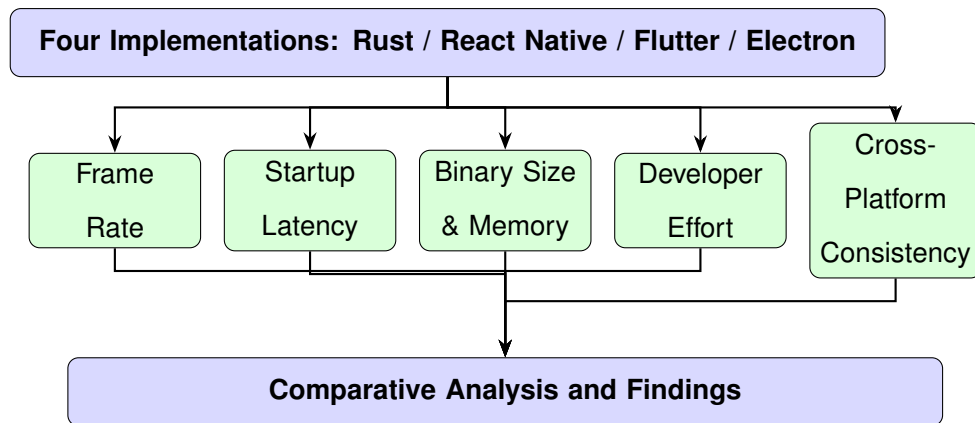


Figure 4: Planned evaluation structure. The scope of comparison frameworks may be adjusted during implementation.

Rendering performance: Frame rate during active UI interaction will be measured using platform frame timing APIs. The target for the Rust/GPUI implementation is 120 fps; comparative measurements will be taken for each baseline framework included in the final evaluation scope.

Startup latency: Time from process launch to first rendered frame, measured using system-level instrumentation on each platform. This metric directly reflects the cost of runtime initialisation, with Electron’s bundled Chromium expected to impose the largest overhead.

Binary size and memory consumption: Compiled binary sizes and resident set size during active use will be recorded on each platform. These metrics are of particular practical relevance given Electron’s documented resource overhead (Lissens, 2023).

Developer effort: The lines of code required to implement each application in each framework, the number of framework-specific concepts required, and the subjective difficulty of achieving correct behaviour will be documented. For the Rust implementation, this will reflect the author’s first-hand experience. For the comparison frameworks, this will reflect the quality and completeness of the AI-generated output and any corrections required.

Cross-platform consistency: Visual and behavioural differences between platform targets will be documented for each framework, assessing the degree to which a single codebase achieves genuinely consistent behaviour in practice.

Note on scope: The intention is to compare against all four implementations: Rust, React Native, Flutter, and Electron. In practice, the number of comparison frameworks included in

the final evaluation may be adjusted depending on time constraints and the complexity of producing reliable AI-generated implementations. React Native and Electron are considered the highest-priority comparisons given their relevance to the web-technology-based approach this project challenges. Any scope adjustment will be documented and justified in the final report.

4 Plan of Work

The following table outlines the key phases of the project from the submission of this interim report to the final submission in August 2026.

Table 1: Project plan of work

Phase	Period	Activities
1. Foundation	May – June 2026	Set up Rust toolchain and GPUI dependency; establish cross-platform build pipeline for Linux, macOS, and Windows; implement core framework primitives including windowing, layout, component model, and state management.
2. Calculator App	June 2026	Implement calculator application using the framework; produce equivalent implementations in React Native, Flutter, and Electron using AI-assisted code generation; run initial performance benchmarks across all four implementations.
3. Memo App	June – July 2026	Implement memo application in the Rust framework; produce equivalents in comparison frameworks; extend benchmarks to cover text rendering and list scrolling performance.
4. Evaluation	July 2026	Systematic benchmarking across all five evaluation dimensions; collect and analyse quantitative data; document developer-experience observations for each framework.
5. Write-up	July – August 2026	Write final report; revise literature review in light of implementation findings; prepare appendices including AI generation logs and code deliverable.
6. Submission	August 2026	Final submission of report and code deliverable.

Table 2: Project timeline

Phase	May	June	July	August
1. Foundation	•	•		
2. Calculator App		•	•	
3. Memo App			•	•
4. Evaluation			•	•
5. Write-up			•	•
6. Submission				•

5 Risk Assessment

Several risks to project delivery have been identified and mitigated as follows.

GPUI API instability: GPUI is pre-1.0 and breaking changes between versions are noted in the documentation (Zed Industries, 2025b). This is mitigated by pinning a specific GPUI version at the start of the implementation phase and only upgrading deliberately.

Windows platform support: GPUI's Windows support is less mature than its macOS and Linux targets as of mid-2025. If full Windows support cannot be achieved within the project timeline, the evaluation will be conducted on the two supported platforms with Windows treated as a documented limitation rather than a failure.

Framework scope: Building a general-purpose GUI framework is a large undertaking. The scope is deliberately constrained to what is necessary to implement the two demonstration applications and will not attempt to match the feature breadth of Flutter or React Native.

Comparison implementation quality: AI-generated code may require substantial correction for certain frameworks. If this occurs, the corrections are documented as part of the developer-experience evaluation rather than being treated as a problem. This is consistent with the agreed methodology.

6 Supervision Meetings

Table 3: Record of supervision meetings

Date	Format	Topics Discussed
[06-02-26]	Online	Initial project concept discussion; identification of Rust-based GUI development as the project domain; agreement on a development-led approach.

Regular fortnightly supervision meetings are planned from the submission of this report through to the final submission in August 2026. Meeting notes will be maintained and appended to the final report.

7 Conclusion

This interim report has established the research context, aims, and methodology for a development-led honours project investigating the use of Rust as the foundation for a cross-platform native GUI framework. The literature review demonstrates that Rust’s formally verified memory safety (Jung et al., 2020), zero-cost abstractions (Klabnik and Nichols, 2019), and deterministic memory model (McCarthy et al., 2022) provide a compelling foundation for high-performance GUI development. The acceptance of Rust into the Linux kernel (The Linux Kernel Organization, 2022) confirms the language’s maturity at the systems level, and GPUI provides a technically sound rendering substrate on which the framework can be built (Zed Industries, 2025d).

The project will produce two concrete software deliverables, a calculator and a memo application, implemented using the Rust framework and evaluated against AI-generated equivalent implementations in React Native, Flutter, and Electron. The findings are expected to provide the first academically grounded empirical evaluation of Rust-based cross-platform GUI development, contributing to a research area that is growing rapidly in industry but has received limited formal scholarly attention.

References

- Bugden, W. and Alahmar, A. (2022). Rust: The programming language for safety and performance. *arXiv preprint arXiv:2206.05503*.
- Crichton, W., Patrignani, M., Agrawala, M., and Hanrahan, P. (2022). Learning and programming challenges of Rust: A mixed-methods study. In *Proceedings of the 44th International Conference on Software Engineering*, pages 1234–1244. ACM.
- De Macedo, J., Abreu, R., Pereira, R., and Saraiva, J. (2022). WebAssembly versus JavaScript: Energy and runtime performance. In *2022 IEEE 19th International Conference on Software Architecture Companion*. IEEE.
- Ernerfeldt, E. (2024). egui: An immediate mode GUI in Rust. GitHub.
- Ferdowsi, K. (2023). The usability of advanced type systems: Rust as a case study. *arXiv preprint arXiv:2301.02308*.
- Fulton, K. R., Chan, A., Votipka, D., Hicks, M., and Mazurek, M. L. (2021). Benefits and drawbacks of adopting a secure programming language: Rust as a case study. In *Proceedings of the 17th Symposium on Usable Privacy and Security*, pages 597–616. USENIX.
- Google LLC (2025). Flutter: Build apps for any screen. flutter.dev.
- Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A., and Bastien, J. (2017). Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 185–200. ACM.
- Heise Online (2026). Zed 1.0: GPU-accelerated open-source editor with AI support. heise.de.
- Hevner, A. R., March, S. T., Park, J., and Ram, S. (2004). Design science in information systems research. In *MIS Quarterly*, volume 28, pages 75–105. Management Information Systems Research Center.
- Horn, M. (2025). A 2025 survey of Rust GUI libraries. boringcactus.com.

- Ivanov, N. (2022). Is Rust C++-fast? benchmarking system languages on everyday routines. *arXiv preprint arXiv:2209.09127*.
- Jung, R., Jourdan, J.-H., Krebbers, R., and Dreyer, D. (2020). RustBelt: Securing the foundations of the Rust programming language. *Journal of the ACM*, 67(4):26:1–26:44.
- Kan, S., Sanán, D., Lin, S.-W., and Liu, Y. (2018). K-Rust: An executable formal semantics of Rust. *arXiv preprint arXiv:1804.07608*.
- Klabnik, S. and Nichols, C. (2019). *The Rust programming language*. No Starch Press, San Francisco, CA.
- Lattuada, A., Hance, T., Cho, C., Brun, M., Subasinghe, I., Zhou, Y., Howell, J., Parno, B., and Hawblitzel, C. (2023). Verus: Verifying Rust programs using linear ghost types. In *Proceedings of the ACM on Programming Languages*, volume 7. ACM.
- Lissens, R. (2023). Electron apps are memory hogs: A deep dive. Technical blog post.
- LogRocket Blog (2024). Exploring Zed, an open-source code editor written in Rust. blog.logrocket.com.
- Matsakis, N. D. and Klock, F. S. (2014). The Rust language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology*, pages 103–104. ACM.
- McCarthy, J. et al. (2022). Benchmarking and learning garbage collection delays for resource-restricted graphical user interfaces. *Software: Practice and Experience*, 52(6):1435–1456.
- Meta Platforms, Inc. (2025). React Native: Learn once, write anywhere. reactnative.dev.
- Microsoft Security Response Centre (2019). A proactive approach to more secure code. Technical report, Microsoft Corporation.
- OpenJS Foundation (2025). Electron: Build cross-platform desktop apps with JavaScript, HTML, and CSS. electronjs.org.
- Szekeres, L., Payer, M., Wei, T., and Song, D. (2013). SoK: Eternal war in memory. In *2013 IEEE Symposium on Security and Privacy*, pages 48–62. IEEE.
- The Linux Kernel Organization (2022). Rust support merged into Linux kernel 6.1. kernel.org.

Torvalds, L. (2022). [GIT PULL] rust for 6.1. Linux Kernel Mailing List.

Watt, C. (2018). Mechanising and verifying the WebAssembly specification. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 53–65. ACM.

Zed Industries (2024). Zed is now open source. zed.dev blog.

Zed Industries (2025a). Awesome GPUI: Projects built with or for GPUI. GitHub.

Zed Industries (2025b). gpui — Zed’s GPU-accelerated UI framework. crates.io.

Zed Industries (2025c). GPUI: A hybrid immediate and retained mode, GPU-accelerated UI framework for Rust. gpui.rs.

Zed Industries (2025d). zed/crates/gpui/README.md. GitHub.